FactoryBench: Benchmarking Machine Understanding via Question-Answering

Yanis Merzouki
ETH Zurich
ymerzouki@student.ethz.ch

Coral Izquierdo Muniz
Forgis AG
coral.izquierdo@forgis.com

Matei Ignuta-Ciuncanu
Imperial College London
mic18@imperial.ac.uk

Jonas Petersen
Forgis AG, ETH Zurich
jonas.petersen@forgis.com

Abstract

We introduce **FactoryBench**, a benchmark for evaluating LLM agents on their understanding of machine behavior over industrial time-series data. Our contributions are threefold. First, we propose a scalable framework for generating machine-understanding question-answering tasks, built around 80 structured question templates spanning diverse reasoning scenarios. Second, we present **FactoryWave**, a dense multivariate dataset generated from one-armed robotic systems, including both collaborative and industrial manufacturing robots. Third, we construct FactoryBench as a large-scale Q&A dataset for time-series understanding in LLM agents, grounded in FactoryWave as well as open-source robotics datasets and simulations. Together, these components provide a rigorous testbed for evaluating reasoning, causal understanding, and decision support over real and simulated industrial signals.

[TODO: check the exact number when level 4 implemented]

1 Introduction

Modern industrial systems generate large volumes of multivariate time-series data from sensors, actuators, and controllers. In robotic manufacturing settings—such as Universal Robots UR-series collaborative arms deployed on automotive assembly lines or KUKA industrial manipulators in electronics packaging—these signals encode joint states, torques, forces, velocities, contact events, task phases, and fault indicators. Extracting actionable knowledge from such data is central to monitoring, diagnostics, anomaly detection, and decision support.

Traditional time-series models excel at narrow tasks such as prediction, classification, and anomaly detection Zhou et al. [2021], Su et al. [2019], Audibert et al. [2020]. However, these models are often specialized and difficult to interpret. They typically output labels or numeric predictions without explicit reasoning or higher-level explanations, and generalize poorly outside of the specific task they are trained on Chalapathy and Chawla [2019], Lavin and Ahmad [2015], Zeng et al. [2023], which limits their direct utility for automated engineering decision-making.

Large language models, in contrast, exhibit strong general reasoning abilities and can produce structured explanations in technical contexts Wei et al. [2022], Yao et al. [2023b]. They can integrate textual information, follow logical chains, and generate interpretable outputs. Yet, when applied directly to dense numerical time series, general LLMs still underperform without adaptation Gruver et al. [2023], Jin et al. [2024]. In contrast, specialized transformer-based architectures have become increasingly effective for time-series forecasting and representation learning Vaswani et al. [2017], Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], Woo et al. [2024], Das et al. [2024].

38th Conference on Neural Information Processing Systems (NeurIPS 2024).

A promising paradigm is to build LLM agents equipped with specialized tools, including time-series models and signal-processing modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These agents can combine language reasoning with domain-specific computation. Nevertheless, evaluating whether such systems truly reason about machine behavior—rather than pattern-matching on surface textual cues—remains challenging. By machine behavior understanding we mean the ability to:

I. interpret the current operational state from raw multivariate signals

II. predict the effect of an intervention on the system

III. reason counterfactually about what would have happened under alternative histories

IV. recommend remedial actions grounded in physical and engineering constraints

Current LLMs typically fail on II, III and IV because they lack access to calibrated dynamics and cannot reliably ingest long, dense numerical streams. Existing benchmarks mostly target textual reasoning, code generation, or generic multimodal understanding Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024], leaving a gap in rigorous evaluation of machine-behavior reasoning over industrial telemetry.

To address this gap, we propose FactoryBench, a benchmark designed to evaluate machine-behavior reasoning in time series models and LLMs. FactoryBench is grounded in FactoryWave, a dense multivariate time-series dataset collected from one-armed robotic systems, including collaborative and industrial manufacturing robots. FactoryWave captures synchronized setpoint, context, feedback, and effort signals at [TODO: add frequency maybe?].

We also introduce a scalable question-generation framework composed of 80 structured templates spanning a wide range of reasoning types and applicable to general machine data flows. These templates enable systematic construction of question-answering tasks that probe state understanding, intervention reasoning, counterfactual analysis, and decision-making in machine contexts. Using this framework and the FactoryWave dataset, we build FactoryBench, a large-scale Q&A dataset specifically designed to evaluate time-series understanding and engineering reasoning in LLM agents.

By bridging the gap between general language reasoning and specialized time-series modeling, FactoryBench provides a principled foundation for studying machine-behavior reasoning in industrial environments.

Add details about FactoryWave, like frequency, number of episodes, tasks, anomaly types, etc.

[TODO: check the exact number when level 4 implemented]

2 Related work

2.1 LLM agents and tool use

Recent advances in LLM reasoning include prompting and deliberative strategies (e.g., Chain-of-Thought and Tree-of-Thought) that improve multi-step performance Wei et al. [2022], Yao et al. [2023b], alongside tool-augmented paradigms such as Toolformer and ReAct that enable grounded computation through external modules Schick et al. [2023], Yao et al. [2023a], Qin et al. [2023]. These ideas motivate industrial agent design where symbolic reasoning must be combined with numerical workflows.

2.2 Time-series learning for industrial signals

Time-series modeling has advanced through transformer-based architectures (Informer, Autoformer, FEDformer, PatchTST) Zhou et al. [2021], Wu et al. [2021], Zhou et al. [2022], Nie et al. [2023], strong alternative baselines such as TFT and N-BEATS Lim et al. [2021], Oreshkin et al. [2020], and critical comparisons against simpler linear models Zeng et al. [2023]. Foundation-model directions (e.g., Chronos) further explore transfer and zero/few-shot adaptation Woo et al. [2024], Das et al. [2024]. In parallel, anomaly and reliability monitoring literature includes OmniAnomaly, USAD, and Deep SVDD Su et al. [2019], Audibert et al. [2020], Ruff et al. [2018], with surveys and benchmarks highlighting persistent gaps in interpretability and root-cause actionability Chalapathy and Chawla [2019], Lavin and Ahmad [2015].

Table 1: Comparison of FactoryBench with existing benchmarks. "Counterfactual" = questions reasoning over alternative histories; "Free-Form" = open-ended answers judged by LLMs; "Novel Dense TS" = contributes a new multivariate high-frequency dataset.

Benchmark	Machine/Robot	Multivariate TS	Size	Counterfactual	Ranking	Numerical	Free-Form	Novel Dense TS
TimeSeriesExam	×	×	∼700	×	✓	×	×	×
ChatTS	×	✓	∼134k	×	✓	✓	×	×
EngineMT-QA	✓	✓	∼110k	×	✓	✓	✓	×
TSAQA	Partial	×	∼210k	×	✓	×	×	×
Time-MQA	×	✓	∼200k	×	✓	✓	✓	×
MTBench	×	✓	?	×	✓	✓	✓	×
QuAnTS	×	✓	∼150k	×	✓	✓	✓	×
CLEVR	×	×	∼1M	×	×	✓	×	×
CLEVRER	×	×	∼305k	✓	×	✓	✓	×
FactoryBench (Ours)	✓	✓	**TBD**	✓	✓	✓	✓	✓

Table 2: Four levels of machine-behavior reasoning in FactoryBench.

Level	Task	Example Question	Ground Truth Source	Commercial Value
1	State	"What's the current of joint 3 now?"	Sensor data	Fleet monitoring
2	Intervention	"If force in joint 3 increases *now* to X, what happens?"	Simulation (present state)	Diagnostic intervention
3	Counterfactual	"If force had increased to X *at t=20ms*, what would have happened?"	Simulation (past state)	Root cause / Capacity planning
4	Decision	"Robot stopped with error C203A. What to do?"	Manual + sensor fusion	Expert-free recovery

2.3 Causality and benchmark gaps

Causal frameworks provide formal tools for interventions and counterfactuals Pearl [2009], Peters et al. [2017], Rubin [1974], while temporal methods such as Granger causality and modern nonlinear causal discovery support directional reasoning in time series Granger [1969], Runge et al. [2019]. Existing broad benchmarks (MMLU, BIG-bench, MMMU) Hendrycks et al. [2021], Srivastava et al. [2023], Yue et al. [2024] and classical time-series resources Laptev et al. [2015], Dau et al. [2019], Wen et al. [2022] do not directly evaluate machine-centered reasoning over industrial telemetry. FactoryBench targets this gap by jointly testing temporal interpretation, causal reasoning, and engineering decision support. Table 1 positions FactoryBench against closely related Q&A and time-series benchmarks along eight axes. The column labels refer to concepts introduced later in the paper: Counterfactual denotes questions that reason about alternative histories (Level 3, Section 3.1); Free-Form denotes open-ended natural-language answers evaluated via an LLM-as-judge protocol (Section 3.2); and Novel Dense TS indicates whether the benchmark contributes a new, high-frequency multivariate dataset rather than relying exclusively on public sources.

3 FactoryBench framework

3.1 Four levels of machine understanding

Our four-tier hierarchy is explicitly modeled on Pearl's *ladder of causation*—association, intervention, and counterfactual reasoning—which has become the standard organizing principle for causal inference and is increasingly adopted as an evaluation axis for machine-learning systems Pearl [2009], Peters et al. [2017]. We extend the three causal rungs with a fourth decision-making rung that reflects how industrial robotic platforms are actually operated: after interpreting state and causal relationships,

operators must select and execute corrective procedures, typically prescribed by vendor manuals (e.g., Universal Robots error-code handbooks). This final rung aligns with the diagnostic-then-act loop that is standard in fault-tolerant control. Grounding the hierarchy in these established principles ensures that each level probes a qualitatively distinct reasoning skill and that failure modes are interpretable in terms of the underlying causal or decision-theoretic primitive.

To systematically evaluate machine-behavior reasoning, we organize question-answering tasks according to this four-tier hierarchy, each probing distinct reasoning capabilities:

- **Level 1: State.** This level assesses the agent's ability to interpret the current state of the machine, including detection of anomalies, identification of operational modes, and recognition of sensor patterns. Questions at this level require accurate extraction and interpretation of time series features.
- **Level 2: Intervention.** At this level, the agent must reason about the consequences of interventions or events occurring at the present timestep that might perturb the distribution of machine states. Tasks include predicting the immediate impact of control actions, diagnosing faults as they arise, and understanding causal relationships in real time.
- **Level 3: Counterfactual.** This level evaluates the agent's ability to reason about hypothetical scenarios, such as the effect of an event or intervention at a previous timestep. Questions require the agent to simulate alternative histories and assess how outcomes would differ under counterfactual conditions, while still considering the history they know.
- **Level 4: Decision Making.** The highest level encompasses complex decision-making tasks, where the agent must generate a sequence of actions or recommendations based on the time series and a prompt. This includes troubleshooting, optimization, and planning, requiring integration of state interpretation, causal reasoning, and goal-directed synthesis.

By structuring Q&A tasks along these four levels, FactoryBench enables rigorous and granular assessment of machine-behavior reasoning, from basic state recognition to advanced decision support.

Design principle: compositional dependency. Each level presupposes the competencies of the previous one. Intervention reasoning (Level 2) requires correctly identifying the current state (Level 1): one cannot predict the effect of perturbing a joint torque without first reading the joint's current operating regime. Counterfactual reasoning (Level 3) additionally requires being able to simulate interventions (Level 2) in a modified history. Decision-making (Level 4) requires combining all three to map observed symptoms to remedial procedures. This compositional structure mirrors Pearl's ladder of causation Pearl [2009], in which each higher rung strictly subsumes the inferential machinery of the lower ones.

3.2 Answer formats

FactoryBench uses four answer formats, chosen to balance evaluation rigor with scalability. Each format supports deterministic scoring (with the exception of free form), and has been chosen in order to make questions very easily checkable, but also hard to answer in the absence of correct reasoning.

- **Multi-Select MC.** The model is presented with four independent statements (A–D) about the outcome of an event or intervention, and must select any subset of statements. Evaluation gives a score of 1 for exact match, 0.5 for 1 mistake, and 0 otherwise.

  A uniformly random guesser selects each statement independently with probability $1/2$; the probability of an exact match on four statements is $1/16$, the probability of exactly one mistake is $4/16$, yielding an expected score of $1 \cdot \frac{1}{16} + 0.5 \cdot \frac{4}{16} = 3/16 \approx 0.188$. We report this baseline alongside model performance so that reasoning gains are measured against the chance ceiling.
- **Ranking.** The model is given four time-series segments or outcomes (A–D) and must order them according to a specified criterion (e.g., severity of deviation, magnitude of a signal). The answer is a permutation string (e.g., `DABC`). Evaluation uses exact-match rate and Kendall's τ rank correlation against the ground-truth ordering.
- **Tensor.** The model must output at least one specific scalar value—such as the expected sensor reading following an intervention—with no multiple-choice scaffolding. The answer

Table 3: Open-source datasets incorporated into FactoryBench.

Dataset	Robot	Size	Frequency	Task	# Anomaly Types
AURSAD	UR3e	?	100 Hz	Screwing	4
voraus-AD	UR5	?	100/500 Hz	Pick and Place	12

is a list of floating-point numbers (possibly one). Evaluation is done on each scalar separately, and uses mean absolute percentage error (MAPE) and a threshold-based accuracy metric (prediction within $\pm k\%$ of ground truth), with partial points given to all correct scalars given.

- **Free-Form.** The model produces an open-ended natural language response such as a diagnosis, recommended action sequence, or causal explanation. Because no single deterministic answer exists, evaluation is performed via an LLM-as-judge voting protocol: three independent frontier LLMs each compare the model's response against the ground-truth reference and cast a verdict of **Wrong** (0), **Neutral** (0.5), or **Correct** (1). The final score is the majority vote across the three judges, with ties broken by averaging.

3.3 Time series data and causal schema (SCE)

To ensure that the dataset structure itself encodes causality, FactoryBench organizes all time-series signals into three causal groups, answering the core question: *"What was the machine told to do, and what did it actually do?"*

- **Setpoint:** The controller's command—target position, velocity, and acceleration per axis.
- **Context:** Physical conditions affecting behavior—payload, temperature, material properties. Split into *static* (episode metadata) and *dynamic* (time-series columns).
- **Effort + Feedback:** The machine's response—motor current (effort), actual position (feedback), vibration, and acoustic emission.

This structure enables a universal fault definition. Under healthy operation, the measured Effort+Feedback response $\mathbf{y}_t$ is defined as a direct function of the Setpoint command $\mathbf{S}$ and the Context $\mathbf{C}_t$:

$$\mathbf{y}_t = f(\mathbf{S}, \mathbf{C}_t)$$

where f represents the nominal plant dynamics (inverse-dynamics-plus-controller transfer) calibrated per robot. Faults manifest as structured deviations from this expected baseline—specifically, residuals $\mathbf{y}_t - f(\mathbf{S}, \mathbf{C}_t)$ that form a clear, fault-specific spatiotemporal pattern. Because FactoryBench supplies the paired $(\mathbf{S}, \mathbf{C}_t, \mathbf{y}_t)$ streams, this residual can be computed directly, establishing a single operational definition of a fault that applies across machines and tasks.

This data is sourced from:

- **FactoryWave:** A custom dataset generated from one-arm robotic platforms executing canonical industrial tasks (e.g., pick-and-place) under varied conditions and systematically injected anomalies.
- **Open Source Datasets:** Adapted datasets such as Aursad and Vorausad, offering diverse industrial scenarios and preprocessed to conform to the unified episode structure.
- **Simulations:** Synthetic time-series data generated from simulated robotic systems to enable controlled experimentation, ablation studies, and validation across both physical and virtual domains.

The two open-source datasets currently integrated into FactoryBench are summarized in Table 3; both provide the full setpoint–context–effort triplet at 100 Hz or higher and have been relabelled to conform to our unified episode schema.

4 Reliable Q&A generation (key contribution)

4.1 At scale generation via extensive labelling

Scalable ground-truth generation is the central challenge of any Q&A benchmark grounded in raw sensor data. FactoryBench addresses this by coupling a structured labelling ontology with a context-free grammar (CFG)-style template system, designed by PhD-level experts in robotics. Rather than annotating individual questions by hand, each template is parameterized: concrete values are filled at generation time from the episode data and its associated labels. The context time series used to fill the variables of the question is sampled uniformly by datasource (sim vs open source vs FactoryWave), dataset (if looking at open source), experiment, difficulty, length and then placement in that order. This yields a combinatorial expansion from a small set of carefully designed templates into a large, diverse question pool.

Guarding against template shortcuts. A natural concern is that template-generated benchmarks can be solved by learning the surface structure of templates rather than the underlying reasoning. Our primary defense is structural: each template admits a combinatorially large instantiation space over signal names, timestamps, prediction horizons, payload conditions, and injected-fault types, so two questions sharing the same template rarely share more than a fraction of their variable slots. Crucially, the correct answer is never recoverable from the question text alone—it is determined by the paired time series, which varies across instantiations. We further characterize the semantic coverage of the generated pool in Section 4.4.

Variable sampling. Each question template contains multiple variable slots that are filled at generation time via variable-specific sampling distributions. Continuous variables—such as signal names, timestamps, prediction horizons, and numerical thresholds—are sampled directly from the episode data or drawn from predefined distributions. Discrete variables—such as tasks, anomaly types, and root causes—are sampled from curated vocabularies. These vocabularies were seeded from the label sets of the open-source datasets incorporated in FactoryBench (AURSAD, voraus-AD) and extended to cover additional operationally realistic cases that arise in FactoryWave. While the extension relied on expert judgment, the resulting vocabularies are released in full as part of the benchmark, so that every question instantiation is inspectable and reproducible. This separation between template structure and sampled content is what allows a small number of hand-authored templates to generate a large and semantically diverse question pool.

Template design. Each of the 80 question templates was manually authored to probe one specific reasoning capability at the appropriate level of the hierarchy, while remaining general enough to admit a wide range of concrete instantiations. Templates are parameterized over episode segments, signal names, event descriptions, timestamps, and predicted values. Answer options for multi-select questions are drawn from a shared pool of verifiable statements, each paired with a rule that can be evaluated deterministically against the time series, given the densely labelled data. This design ensures that ground truth is never imputed or inferred—it is computed directly from labeled episode data—making the benchmark both reliable and fully reproducible.

[TODO: check the exact number when level 4 implemented]

4.2 Density of FactoryWave

4.3 Adapting labelling to open datasets and simulations

4.4 Dataset diversity & quality assurance

A critical risk in template-generated benchmarks is a lack of semantic diversity, leading models to memorize structural patterns rather than perform true reasoning. To ensure our dataset evaluates robust machine-behavior reasoning, we utilize the **Vendi Score** Friedman and Dieng [2023] on the embeddings of our Q&A pairs to measure and maximize effective population diversity.

We iteratively evaluate dataset diversity across three primary axes during generation:

- **Low Diversity (Parameter Variation):** Varying only parameters like time windows and joint indices yields a low Vendi Score, confirming that simple parameter randomization is insufficient.

- **Moderate Diversity (Format Variation):** Semantically identical questions expressed across different formats (Open-ended, Multiple Choice, True/False) measurably increase the effective diversity.
- **High Diversity (Template & Type Variation):** Introducing mixed reasoning templates across levels (e.g., kinematic comparison, derivative estimation like friction/acceleration, anomaly detection) drives the highest Vendi Score. By enforcing high Vendi Scores across our generated subsets, we ensure the benchmark tests versatile analytical capabilities.

4.5 Pipeline overview

Some practical questions: How can people employ the benchmark? Where can it be found. Will it be maintained and expanded? Will incorrect labels be corrected? Is there a private dataset too? Are parts of the dataset designated for training and parts for testing?

Access and maintenance. FactoryBench and FactoryWave are released under a license and distributed via a public Hugging Face repository. The release includes the episode data, the generator source code, the full set of 80 question templates with their paraphrase banks, and all LLM-as-judge prompts. We provide a versioned release track (e.g., `v1.0`, `v1.1`) so that reported numbers always refer to a fixed benchmark state; corrections to labels or templates are published as minor-version updates with a public changelog, and the exact version used in any evaluation is stamped into every result file.

check license

5 Experiments

5.1 Models to evaluate

Frontier LLMs:

- GPT-4o, GPT-4-Turbo
- Gemini-2.5-Pro, Gemini-2.5-Flash
- Claude-3.5-Sonnet, Claude-3-Opus

Specialized Methods:

- Time-series encoders + LLM (Chronos, Moirai)
- Multimodal industrial models (FD-LLM)
- RAG with manual retrieval

Baselines:

- Random
- Rule-based (threshold detection)
- Human expert (ceiling)

5.2 Experimental configurations

Beyond out-of-the-box evaluation, we investigate how different interventions improve performance:

- **Finetuning:** We optionally finetune open-source models on the FactoryBench Q&A dataset to establish the limit of specialized training versus general reasoning.
- **Tool-Augmented Agents:** The best-performing foundational models are granted access to external time-series prediction and anomaly detection modules, testing the synergy between LLM reasoning and domain-specific computation.

5.3 Results (expected)

Hypothesis: Semantic priors (ManualsGraph) will show largest gains at Level 4.

Table 4: Expected performance across levels (accuracy %).

Level	Random	Current LLMs	Target (w/ prior)	Human Expert
1	50%	85–95%	98%	99%
2	20%	60–75%	85%	95%
3	10%	35–50%	70%	90%
4	5%	15–30%	45%	80%

6 Discussion

6.1 Why this benchmark matters

For researchers: First rigorous evaluation of machine understanding across reasoning levels.

For practitioners: Quantifies which AI capabilities are ready for deployment as AI engineers.

For the field: Defines "machine understanding" operationally via Q&A.

6.2 Limitations

- Single machine family (by design, for depth)
- Simulation-to-real gap in synthetic episodes
- Reliance on LLMs-as-Judge for evaluation of Free-Form questions

6.3 Relation to FactoryNet

This benchmark evaluates on a **subset** of FactoryNet. The full dataset (50k+ episodes, 200k+ Q&A) will be released separately with a focus on *training* industrial world models, not just evaluation.

7 Conclusion

FactoryBench introduces a systematic approach to evaluating machine understanding via Q&A. By focusing deeply on one machine family and providing reliable answer generation at scale, we enable rigorous comparison of methods across four levels of reasoning—from basic state identification to procedure generation grounded in technical manuals.

Our physical validation protocol closes the loop from benchmark to reality, addressing the fundamental question: can AI systems that excel at industrial Q&A actually help in the real world?

References

Julien Audibert et al. Usad: Unsupervised anomaly detection on multivariate time series. *KDD*, 2020.

Raghavendra Chalapathy and Sanjay Chawla. Deep learning for anomaly detection: A survey. *arXiv preprint*, 2019.

Abhimanyu Das et al. Time series foundation models and forecasting: A survey. *arXiv preprint*, 2024.

Hoang Anh Dau et al. The ucr time series classification archive. *IEEE/CAA Journal of Automatica Sinica*, 2019.

Dan Friedman and Adji Bousso Dieng. The vendi score: A diversity evaluation metric for machine learning. *TMLR*, 2023.

Clive W. J. Granger. Investigating causal relations by econometric models and cross-spectral methods. *Econometrica*, 1969.

Nate Gruver, Marc Finzi, Shikai Qiu, and Andrew Gordon Wilson. Large language models are zero-shot time series forecasters. *NeurIPS*, 2023.

Dan Hendrycks et al. Measuring massive multitask language understanding (mmlu). *ICLR*, 2021.

Ming Jin et al. Time-llm: Time series forecasting by reprogramming large language models. *ICLR*, 2024.

Nikolay Laptev, Saeed Amizadeh, and Ian Flint. Generic and scalable framework for automated time-series anomaly detection. *KDD*, 2015.

Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the numenta anomaly benchmark. *IEEE ICMLA*, 2015.

Bryan Lim et al. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *IJF*, 2021.

Yuqi Nie et al. A time series is worth 64 words: Long-term forecasting with transformers (patchtst). *ICLR*, 2023.

Boris N. Oreshkin et al. N-beats: Neural basis expansion analysis for interpretable time series forecasting. *ICLR*, 2020.

Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd edition, 2009.

Jonas Peters, Dominik Janzing, and Bernhard Schölkopf. *Elements of Causal Inference*. MIT Press, 2017.

Yujia Qin et al. Tool learning with foundation models. *arXiv preprint*, 2023.

Donald B. Rubin. Estimating causal effects of treatments in randomized and nonrandomized studies. *Journal of Educational Psychology*, 1974.

Lukas Ruff et al. Deep one-class classification. *ICML*, 2018.

Jakob Runge et al. Detecting and quantifying causal associations in large nonlinear time series datasets. *Science Advances*, 2019.

Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023.

Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models (big-bench). *TMLR*, 2023.

Ya Su et al. Robust anomaly detection for multivariate time series through stochastic recurrent neural network (omnianomaly). *KDD*, 2019.

Ashish Vaswani et al. Attention is all you need. *NeurIPS*, 2017.

Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *NeurIPS*, 2022.

Qingsong Wen et al. Transformers in time series: A survey. *IJCAI*, 2022.

Gerald Woo et al. Chronos: Learning the language of time series. *arXiv preprint*, 2024.

Haixu Wu et al. Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting. *NeurIPS*, 2021.

Shunyu Yao et al. React: Synergizing reasoning and acting in language models. *ICLR*, 2023a.

Shunyu Yao et al. Tree of thoughts: Deliberate problem solving with large language models. *NeurIPS*, 2023b.

Xiang Yue et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. *CVPR*, 2024.

Ailing Zeng et al. Are transformers effective for time series forecasting? *AAAI*, 2023.

Haoyi Zhou et al. Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI*, 2021.

Tian Zhou et al. Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting. *ICML*, 2022.

A Q&A pair structure

Level 2 Example (Intervention):


```
{
  "id": "4fabb9ab-e476-48ff-812d-8fc33bb63ac3",
  "level": 2,
  "template_id": 2,
  "template_type": "intervention_outcome",
  "question": "If a continual increase of feedback_tcp_speed_5
    lasting 15 timesteps happened at the current timestep
    (T=4640ms), what would most likely happen next?
    Answer only with a 4 letter string using F and T ...",
  "options": {
    "A": "No significant effect: safety stays normal ...",
    "B": "Following the event, command and measured TCP ...",
    "C": "Following the event, the system remains ...",
    "D": "Following the event, robot current increases ..."
  },
  "answer": "FTTF",
  "provenance": {
    "dataset": "inter_aursad",
    "episode": "experiment_1",
    "subseries_start_index": 131,
    "subseries_length": 62
  },
  "context": {...}
}
```


B Benchmark inference cost

A practical concern when deploying large-scale Q&A benchmarks is the monetary cost of running inference across all questions. We derive a simple cost model and tabulate estimates for three representative frontier models under batch-API pricing.

B.1 Cost model

Let Q denote the total number of question–answer pairs in the benchmark. For each question q, the API call consumes $t_{\text{in}}^{(q)}$ input tokens (prompt, context time series, and answer options) and $t_{\text{out}}^{(q)}$ output tokens (model response). We denote the per-token prices as p_{in} and p_{out} respectively (in USD per token). The total cost is:

$$C = \sum_{q=1}^{Q} \left(p_{\text{in}} \cdot t_{\text{in}}^{(q)} + p_{\text{out}} \cdot t_{\text{out}}^{(q)} \right). \tag{1}$$

Since individual token counts vary by question, we work with corpus-level averages. Let $\bar{t}_{\text{in}}$ and $\bar{t}_{\text{out}}$ denote the mean input and output tokens per question. Then:

$$C \approx Q \cdot \left(p_{\text{in}} \cdot \bar{t}_{\text{in}} + p_{\text{out}} \cdot \bar{t}_{\text{out}} \right). \tag{2}$$

It is convenient to express prices per million tokens. Writing P_{in} and P_{out} for the price in USD per million tokens:

$$C = Q \cdot \left(\frac{P_{\text{in}} \cdot \bar{t}_{\text{in}} + P_{\text{out}} \cdot \bar{t}_{\text{out}}}{10^6} \right). \tag{3}$$

B.2 Assumptions

Input tokens. Each question payload averages approximately 30 KB, comprising the system prompt, question text, serialized time-series context window, and answer options. At an empirical ratio of 0.25 tokens per byte, this yields:

$$\bar{t}_{\text{in}} = 30{,}000 \text{ bytes} \times 0.25 \text{ tokens/byte} = 7{,}500 \text{ tokens}.$$

Output tokens. The majority of FactoryBench answer formats—Multi-Select MC (`FTTF`), Ranking (`DABC`), and Tensor (a scalar)—require only $\sim$2 output tokens. Free-form answers, which constitute at most 10% of the benchmark, produce longer responses ($\sim$300 tokens). The weighted average is therefore:

$$\bar{t}_{\text{out}} = 0.9 \times 2 + 0.1 \times 300 = 31.8 \approx 32 \text{ tokens}.$$

Batch constraints. Most batch APIs impose a maximum request file size of 50 MB. At 30 KB per question, each batch accommodates $\lfloor 50{,}000/30 \rfloor \approx 1{,}666$ questions. A full benchmark run of $Q = 200{,}000$ therefore requires $\lceil 200{,}000/1{,}666 \rceil = 120$ batches. This affects throughput scheduling but not per-token pricing.

All models are evaluated via their respective **batch processing APIs**, which offer reduced pricing with longer turnaround times (typically up to 24 hours). This is the natural choice for offline benchmark evaluation.

B.3 Model pricing

Table 5 summarizes the batch-API pricing for three frontier models as of April 2026.

Table 5: Batch-API pricing for selected frontier models (USD per million tokens, April 2026).

Model	P_{in} (\$/M tokens)	P_{out} (\$/M tokens)
Claude Opus 4.6 (Anthropic)	2.50	12.50
GPT-4o (OpenAI)	1.25	5.00
Gemini 2.5 Pro (Google)	0.625	5.00

B.4 Sample calculation for $Q = 200{,}000$

Substituting into Equation 3 with $Q = 200{,}000$, $\bar{t}_{\text{in}} = 7{,}500$, and $\bar{t}_{\text{out}} = 32$:

Claude Opus 4.6.

$$\begin{aligned} C_{\text{Opus}} &= 200{,}000 \times \frac{2.50 \times 7{,}500 + 12.50 \times 32}{10^6} \\ &= 200{,}000 \times \frac{18{,}750 + 400}{10^6} \\ &= 200{,}000 \times 0.01915 = \$\mathbf{3{,}830}. \end{aligned} \tag{4}$$

GPT-4o.

$$\begin{aligned} C_{\text{GPT-4o}} &= 200{,}000 \times \frac{1.25 \times 7{,}500 + 5.00 \times 32}{10^6} \\ &= 200{,}000 \times \frac{9{,}375 + 160}{10^6} \\ &= 200{,}000 \times 0.009535 = \$\mathbf{1{,}907}. \end{aligned} \tag{5}$$

Gemini 2.5 Pro.

$$\begin{aligned} C_{\text{Gemini}} &= 200{,}000 \times \frac{0.625 \times 7{,}500 + 5.00 \times 32}{10^6} \\ &= 200{,}000 \times \frac{4{,}687.5 + 160}{10^6} \\ &= 200{,}000 \times 0.0048475 = \$\mathbf{969.50}. \end{aligned} \tag{6}$$

B.5 Summary

Table 6: Estimated benchmark inference cost for $Q = 200{,}000$ questions (batch API).

Model	Input cost	Output cost	**Total cost**	Cost per question
Claude Opus 4.6	\$3,750.00	\$80.00	**\$3,830.00**	\$0.0192
GPT-4o	\$1,875.00	\$32.00	**\$1,907.00**	\$0.0095
Gemini 2.5 Pro	\$937.50	\$32.00	**\$969.50**	\$0.0049

These estimates show that a full benchmark run of 200k questions on a frontier model costs between roughly \$1,000 and \$4,000 using batch APIs. Costs are heavily input-dominated: output tokens contribute less than 5% of the total, since 90% of FactoryBench answers are short structured strings (2 tokens). Standard (non-batch) API pricing is typically $2\times$ higher. Note that these costs are for a *single* evaluation pass; repeated runs (e.g., for variance estimation or ablation studies) scale linearly. The per-question cost (\$0.005–\$0.02) confirms that FactoryBench is economically feasible to evaluate at scale, even on the most expensive frontier models.